

High Level Plan to Migrate 8 TB onprem DB from Old SQL Server to New SQL Server with Minimum DB Downtime.

Here's a **high-level migration plan** for moving an **8 TB on-prem SQL Server database** to a new SQL Server instance with **minimal downtime**, broken into **three phases**: preparation, migration, and cutover.

We'll assume this is an **on-prem** → **on-prem or VM-based SQL Server** migration (not to Azure) and that you want to keep the outage window as short as possible.

Phase 1 – Planning & Preparation

Goal: Identify the right migration method, validate environment, and reduce risk.

Step	Action	Notes
1	Assess Source & Target Environment	Identify SQL Server versions, editions, OS, storage, and compatibility level.
2	Estimate Migration Method	For 8 TB with minimal downtime, use Log Shipping , Database Mirroring (if supported), or Always On Availability Groups for continuous sync before cutover. Backup/restore alone will cause long downtime.
3	Check Network Bandwidth	Large DB replication requires good throughput; consider compression to speed up transfers.
4	Cleanup Database	Remove unused data, archive historical tables, rebuild indexes to reduce size.
5	Run DBCC CHECKDB	Ensure no corruption before migration.
6	Update Statistics	Improves performance after migration.
7	Check Application Dependencies	Test app compatibility with target SQL Server version.

Phase 2 – Pre-Migration (Continuous Data Sync)

Goal: Get the majority of data to the target server **before downtime**.

Option A – Log Shipping

1. Take **full backup** of the source DB.
2. Restore it to the target server **WITH NORECOVERY**.
3. Set up **log shipping** to ship and restore transaction log backups on a schedule (e.g., every 5–10 minutes).

Option B – Database Mirroring (if SQL Server 2008–2016, not available in later versions)

1. Set up mirroring in **High Performance (Asynchronous)** mode.
2. Let it sync until lag is minimal.

Option C – Always On Availability Group

1. Add the target server as a secondary replica.
2. Configure for **asynchronous commit** initially.
3. Let the DB fully synchronize.

Phase 3 – Cutover (Minimal Downtime Window)

Goal: Switch to the new SQL Server with only a short outage.

1. **Notify stakeholders** of downtime start.
2. Stop all application connections to the source DB.
3. Apply final **transaction log backups** (log shipping) or switch AG/mirroring to synchronous commit.
4. Failover or recover the target database:
 - **Log shipping:** Restore last log WITH RECOVERY.
 - **Mirroring / AG:** Perform manual failover.
5. Run **post-migration validation**:
 - DBCC CHECKDB
 - Compare row counts for critical tables
 - Run key application queries
6. Point application connection strings to new SQL Server.
7. Monitor performance & error logs for the first 24–48 hours.

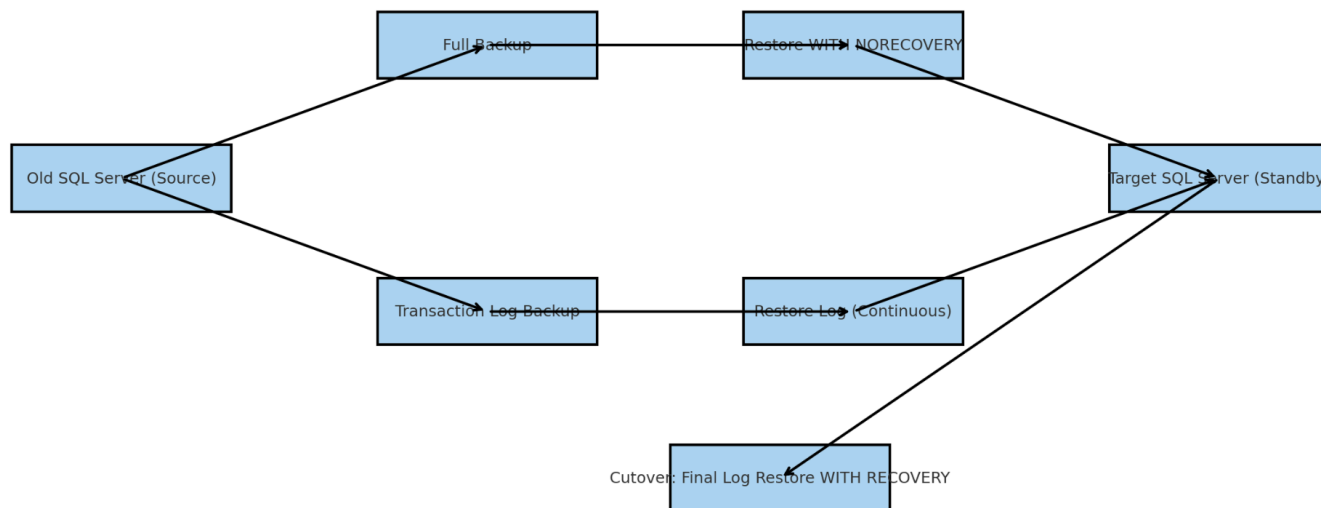
Example Timeline (Using Log Shipping)

Time	Activity
Day –5	Start full backup & restore to target WITH NORECOVERY
Day –4	Set up log shipping; ship logs every 10 mins
Day –1	Verify sync delay is <5 mins
Cutover Window (1–2 hrs)	Stop app, restore final log WITH RECOVERY, validate, point app to new DB

Why This Works for 8 TB

- **Continuous sync** before downtime means only final logs are applied during the outage.
- No need to re-transfer the full DB during downtime.
- Works on standard SQL Server features, no extra licenses for AG (if using log shipping).

Minimal Downtime Migration (Log Shipping Strategy for 8 TB DB)



Here's the **migration strategy diagram** — it shows how you:

1. Take a **full backup** and restore it on the target server in **NORECOVERY** mode.
2. Continuously apply **transaction logs** from the old server to keep it in sync.
3. During the cutover window, apply the **final log** WITH RECOVERY, then switch applications to the new server.